
```
items:  [0, 3, 1, 7, 6, 2, 5, 4]
indices: 0 1 2 3 4 5 6 7
          ^
i = 3      i      ^
j = 5      j
```

The value at index j (which is 2) is less than or equal to the pivot value (which is 4), so swap the values at i and j . The 7 and 2 swap positions. Also, increase i so that it moves to the right. The state of the variables now looks like this:

```
items:  [0, 3, 1, 2, 6, 7, 5, 4]
indices: 0 1 2 3 4 5 6 7
          ^
i = 4      i      ^
j = 6      j
```

The value at index j (which is 6) is not less than or equal to the pivot value (which is 4), so don't swap the values. The state of the variables now looks like this:

```
items:  [0, 3, 1, 2, 6, 7, 5, 4]
indices: 0 1 2 3 4 5 6 7
          ^
i = 4      i      ^
j = 7      j
```

We've reached the end of the partitioning. The index j is at the pivot value (which is always the rightmost value in the range), so let's swap i and j one last time to make sure the pivot is not on the right half of the partition. The 6 and 4 swap positions. The state of the variables now looks like this:

```
items:  [0, 3, 1, 2, 4, 7, 5, 6]
indices: 0 1 2 3 4 5 6 7
          ^
i = 4      i      ^
j = 7      j
```

Notice what is happening with the i index: this index will always receive the values smaller than the pivot value as a result of swapping; then the i